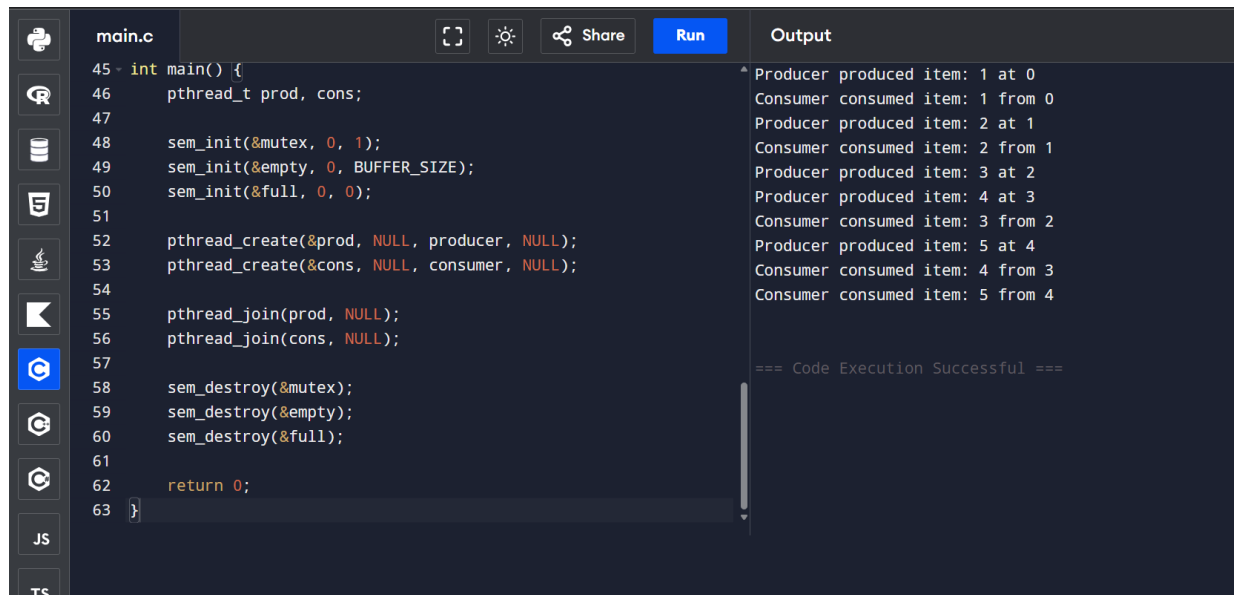


EXPERIMENT 18:

18. Construct a C program to simulate producer-consumer problem using semaphores.



```
main.c
45 - int main() {
46     pthread_t prod, cons;
47
48     sem_init(&mutex, 0, 1);
49     sem_init(&empty, 0, BUFFER_SIZE);
50     sem_init(&full, 0, 0);
51
52     pthread_create(&prod, NULL, producer, NULL);
53     pthread_create(&cons, NULL, consumer, NULL);
54
55     pthread_join(prod, NULL);
56     pthread_join(cons, NULL);
57
58     sem_destroy(&mutex);
59     sem_destroy(&empty);
60     sem_destroy(&full);
61
62     return 0;
63 }
```

Output

```
Producer produced item: 1 at 0
Consumer consumed item: 1 from 0
Producer produced item: 2 at 1
Consumer consumed item: 2 from 1
Producer produced item: 3 at 2
Producer produced item: 4 at 3
Consumer consumed item: 3 from 2
Producer produced item: 5 at 4
Consumer consumed item: 4 from 3
Consumer consumed item: 5 from 4

=== Code Execution Successful ===
```

PSEUDO CODE :

Global Variables:

buffer[BUFFER_SIZE], in = 0, out = 0

Semaphore mutex = 1, empty = BUFFER_SIZE, full = 0

Producer Thread:

Loop:

item = produce_item()

wait(empty) // Decrement available empty slots

wait(mutex) // Lock buffer access

buffer[in] = item

in = (in + 1) % BUFFER_SIZE

signal(mutex) // Unlock buffer

signal(full) // Increment available items

Consumer Thread:

Loop:

```
wait(full)    // Decrement available items
wait(mutex)   // Lock buffer access
item = buffer[out]
out = (out + 1) % BUFFER_SIZE
signal(mutex) // Unlock buffer
signal(empty) // Increment available empty slots
consume_item(item)
```

CODE:

```
#include <stdio.h>

#include <pthread.h>

#include <semaphore.h>

#include <unistd.h>

#define BUFFER_SIZE 5

int buffer[BUFFER_SIZE];

int in = 0, out = 0;

sem_t mutex, empty, full;

void* producer(void* arg) {
    for (int i = 1; i <= 5; i++) {
```

```
sem_wait(&empty);

sem_wait(&mutex);


buffer[in] = i;

printf("Producer produced item: %d at %d\n", i, in);

in = (in + 1) % BUFFER_SIZE;


sem_post(&mutex);

sem_post(&full);

sleep(1);
}

return NULL;
}


void* consumer(void* arg) {
    for (int i = 1; i <= 5; i++) {
        sem_wait(&full);

        sem_wait(&mutex);


        int item = buffer[out];

        printf("Consumer consumed item: %d from %d\n", item, out);

        out = (out + 1) % BUFFER_SIZE;


        sem_post(&mutex);

        sem_post(&empty);
```

```
        sleep(2);
    }
    return NULL;
}

int main() {
    pthread_t prod, cons;

    sem_init(&mutex, 0, 1);
    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);

    pthread_create(&prod, NULL, producer, NULL);
    pthread_create(&cons, NULL, consumer, NULL);

    pthread_join(prod, NULL);
    pthread_join(cons, NULL);

    sem_destroy(&mutex);
    sem_destroy(&empty);
    sem_destroy(&full);

    return 0;
}
```